Report

> ## Task #03 — Buffer Overflow

> **TEAM:** Digital Avengers

> **MEMBRI:** Lorenzo Petrucci, Christian Koscielniak Pinto, Gabriele Chinnici, Wafa Saghir, Amy Antonella Acosta Hugo, Domenico Mendola, Corrado Vacca

Traccia dell'esercizio

L'esercizio richiedeva di analizzare un programma C fornito dal docente e svolgere i seguenti passi: descrivere il funzionamento del programma prima dell'esecuzione, riprodurlo ed eseguirlo nel laboratorio verificando se le ipotesi iniziali erano corrette, e infine modificarlo affinché si verificasse un errore di segmentazione. I bonus richiedevano di inserire controlli di input e creare un menù per far scegliere all'utente tra la versione corretta e quella vulnerabile.

```c
#include <stdio.h>

int main () {

int vector [10], i, j, k;
int swap_var;


printf ("Inserire 10 interi:\n");

for ( i = 0 ; i < 10 ; i++)
    {
    int c= i+1;
    printf("[%d]:", c);
    scanf ("%d", &vector[i]);
    }


printf ("Il vettore inserito e':\n");
for ( i = 0 ; i < 10 ; i++)
        {
        int t= i+1;
        printf("[%d]: %d", t, vector[i]);
    printf("\n");
    }


for (j = 0 ; j < 10 - 1; j++)
    {
    for (k = 0 ; k < 10 - j - 1; k++)
        {
            if (vector[k] > vector[k+1])
            {
            swap_var=vector[k];
            vector[k]=vector[k+1];
```

```
            vector[k+1]=swap_var;
            }
        }
    }
printf("Il vettore ordinato e':\n");
for (j = 0; j < 10; j++)
    {
    int g = j+1;
    printf("[%d]:", g);
    printf("%d\n", vector[j]);
    }

return 0;


}
```

Descrizione prima dell'esecuzione:

Il programma dichiara un array di interi `vector[10]` con dimensione fissa di 10 elementi, e le variabili di supporto `i`, `j`, `k` per i cicli e `swap_var` per lo scambio temporaneo. Il flusso di esecuzione si divide in tre fasi: nella prima fase chiede all'utente di inserire 10 numeri interi tramite `scanf()`, salvandoli nell'array. Nella seconda fase stampa i valori inseriti nell'ordine originale. Nella terza fase ordina i valori in ordine crescente tramite l'algoritmo Bubble Sort e stampa il risultato.

Esecuzione del programma originale

Il programma ha funzionato esattamente come previsto, ovvero ha chiesto 10 numeri, li ha stampati, li ha ordinati con Bubble Sort e li ha ristampati in ordine crescente

```
└─$ ./prova
Inserire 10 interi:
[1]:3
[2]:4
[3]:3
[4]:2
[5]:2
[6]:3
[7]:4
[8]:4
[9]:5
[10]:4
Il vettore inserito e':
[1]: 3
[2]: 4
[3]: 3
[4]: 2
[5]: 2
[6]: 3
[7]: 4
[8]: 4
[9]: 5
[10]: 4
Il vettore ordinato e':
[1]:2
[2]:2
[3]:3
[4]:3
[5]:3
[6]:4
[7]:4
[8]:4
[9]:4
[10]:5
```

Cos'è un Buffer Overflow

Un Buffer Overflow è una vulnerabilità che si verifica quando un programma scrive più dati di quanti un'area di memoria possa contenerne. In C, gli array hanno dimensione fissa dichiarata a compile time. Se il programma tenta di scrivere oltre il limite dell'array, sovrascrive le aree di memoria adiacenti nello stack, che possono contenere altre variabili, indirizzi di ritorno di funzioni, o dati critici del sistema. Questo causa comportamenti imprevedibili, crash del programma, o in scenari reali l'esecuzione di codice arbitrario da parte di un attaccante.

Modifica del programma — bof02.c

Il programma originale è stato modificato per dimostrare concretamente la vulnerabilità. Invece di chiedere i numeri uno alla volta con un limite fisso di 10, la versione modificata legge una intera riga di numeri separati da spazio usando un puntatore diretto all'array. Questo permette all'utente di inserire quanti numeri vuole — senza alcun controllo sulla dimensione dell'array.

Il meccanismo chiave è il ciclo `do...while` che legge numeri finché non trova un `\n`, scrivendo direttamente nelle celle di memoria tramite il puntatore `p + i`. Quando `i` supera 10, il programma scrive oltre i confini dell'array sovrascrivendo la memoria adiacente nello stack.

Inserendo una sequenza casuale di numeri, il programma ha mostrato comportamento anomalo e si è concluso con `zsh: segmentation fault ./bof02`, confermando la presenza del Buffer Overflow.

```
Inserire interi (premi INVIO alla fine):
 6 7 9 7 6 8 9 0 8 6 5 7 8 9 8 7 65 7 89 8 7 6 78 9 8 77 8 9
zsh: segmentation fault  ./bof02
```

```c
#include <stdio.h>int main() {
int vector[10];
int *p = vector;
int i = 0;
int j, k, swap_var;
char c;
printf("Inserire interi (premi INVIO alla fine):\\n");
do {
    if (scanf("%d", (p + i)) == 1) {
        i++;
    }
    while ((c = getchar()) == ' ');
    if (c != '\\n' && c != EOF) ungetc(c, stdin);
} while (c != '\\n' && c != EOF);

printf("\\nIl vettore inserito e':\\n");
for (j = 0; j < i; j++)
    printf("[%d]: %d\\n", j + 1, vector[j]);

for (j = 0; j < i - 1; j++)
    for (k = 0; k < i - j - 1; k++)
        if (vector[k] > vector[k + 1]) {
            swap_var = vector[k];
            vector[k] = vector[k + 1];
            vector[k + 1] = swap_var;
        }

printf("\\nIl vettore ordinato e':\\n");
```

```
for (j = 0; j < i; j++)
    printf("[%d]: %d\\n", j + 1, vector[j]);

return 0;
}
```

Programma modificato con menù — Bonus

Per completare l'esercizio e i bonus è stato sviluppato un programma con menù interattivo composto da tre funzioni.

La funzione `main()` presenta un menù con un ciclo `while` che rimane attivo finché l'utente non sceglie l'opzione 3 (Exit). In base alla scelta chiama la funzione corrispondente. Se l'input non è valido mostra un messaggio di errore e ripresenta il menù.

La funzione `corretto()` implementa il Bubble Sort con validazione completa dell'input. Usa `while (scanf(...) != 1)` per rilevare valori non numerici, `getchar()` per pulire il buffer di input, e stampa i risultati in formato a due colonne per maggiore leggibilità.

La funzione `vulnerabile()` dimostra il Buffer Overflow in modo realistico. Invece di chiedere i numeri uno alla volta, legge una intera riga di numeri separati da spazio in un array dichiarato da soli 10 elementi. Se l'utente incolla più di 10 numeri, il programma scrive oltre i confini dell'array sovrascrivendo la memoria adiacente, causando comportamento imprevedibile e Segmentation Fault.

```
#include <stdio.h>

void corretto(void);
void vulnerabile(void);

int main() {
//MENU INIZIALE
    int scelta = 0;
    while (scelta != 3) {
        printf("\nScegli tra queste tre opzioni: \n");
        printf("(1) Esegui programma CORRETTO (Input validato)\n");
        printf("(2) Esegui programa VULNERABILE (Provoca Segmentation Fault)\n");
        printf("(3) Exit. \n");
        printf("Opzione: ");

        scanf("%d", &scelta);
        if (scelta == 1) {
            corretto();
        } else if (scelta == 2) {
            vulnerabile();
        } else if (scelta == 3) {
            printf("Hai scelto Exit. Arrivederci!\n");
        } else {
            printf("Opzione non valida!\n");
        }
    }
    return 0;
}

//  VERSIONE CORRETTA
```

```c
void corretto(void) {
    int vector[10], i, j, k;
    int swap_var;

    printf("\n--- PROGRAMMA CORRETTO ---\n");
    printf("Inserire 10 interi:\n");

    for (i = 0; i < 10; i++) {
        printf("[%d]: ", i + 1);
        while (scanf("%d", &vector[i]) != 1) {
            printf("Errore: inserire un numero intero!\n");
            printf("[%d]: ", i + 1);
            while (getchar() != '\n');
        }
    }

    printf("\nIl vettore inserito e':\n");
    for (i = 0; i < 5; i++) {
        printf("[%d]: %-5d\t[%d]: %-5d\n", i + 1, vector[i], i + 6, vector[i +
5]);
    }

    for (j = 0; j < 9; j++)
        for (k = 0; k < 9 - j; k++)
            if (vector[k] > vector[k + 1]) {
                swap_var = vector[k];
                vector[k] = vector[k + 1];
                vector[k + 1] = swap_var;
            }

    printf("\nIl vettore ordinato e':\n");
    for (j = 0; j < 5; j++) {
        printf("[%d]: %-5d\t[%d]: %-5d\n", j + 1, vector[j], j + 6, vector[j +
5]);
    }
    printf("\n");
}

//  VERSIONE VULNERABILE
void vulnerabile(void) {
    int vector[10];
    int *p = vector;
    int i = 0;
    int j, k, swap_var;
    char c;

    printf("\n[VULNERABILE] \n Imetti una lunga riga di numeri e premi INVI
O:\n");

    do {
        if (scanf("%d", (p + i)) == 1) {
            i++;
```

```c
        }
        while ((c = getchar()) == ' ');
        if (c != '\n' && c != EOF) ungetc(c, stdin);
    } while (c != '\n' && c != EOF);

    printf("\nHai inserito %d numeri in un'area da 10.\n", i);
    printf("Tentativo di ritorno al menu principale...\n");

    // BUBBLE SORT
    for (j = 0; j < i - 1; j++) {
        for (k = 0; k < i - j - 1; k++) {
            if (vector[k] > vector[k + 1]) {
                swap_var = vector[k];
                vector[k] = vector[k + 1];
                vector[k + 1] = swap_var;
            }
        }
    }

    printf("Vettore elaborato: ");
    for (j = 0; j < i; j++) printf("%d ", vector[j]);
    printf("\n");

    return;
}
```

Dimostrazione del Buffer Overflow

Opzione 1 — Programma corretto: inserendo una lettera invece di un numero, il programma mostra il messaggio di errore e chiede di reinserire il valore, senza bloccarsi.

```
--- PROGRAMMA CORRETTO ---
Inserire 10 interi:
[1]: 4
[2]: 3
[3]: 2
[4]: 3
[5]: 4
[6]: g
Errore: inserire un numero intero!
[6]: 4
[7]: 3
[8]: 4
[9]: h
Errore: inserire un numero intero!
[9]: 4
[10]: 4
```

```
Il vettore inserito e':
[1]: 4          [6]: 4
[2]: 3          [7]: 3
[3]: 2          [8]: 4
[4]: 3          [9]: 4
[5]: 4          [10]: 4

Il vettore ordinato e':
[1]: 2          [6]: 4
[2]: 3          [7]: 4
[3]: 3          [8]: 4
[4]: 3          [9]: 4
[5]: 4          [10]: 4
```

Opzione 2 — Programma vulnerabile: inserendo la sequenza `7 23 4 156 89 12 3 67 445 91 234 8 56 178 33 902 44 7 123 55 678 21 9 333 87` , il programma ha mostrato comportamento anomalo, ovvero i contatori degli slot saltavano in modo imprevedibile e si è concluso con `zsh: segmentation fault ./menu`.

```
Scegli tra queste tre opzioni:
(1) Esegui programma CORRETTO (Input validato)
(2) Esegui programa VULNERABILE (Provoca Segmentation Fault)
(3) Exit.
Opzione: 2

[VULNERABILE]
 Imetti una lunga riga di numeri e premi INVIO:
7 23 4 156 89 12 3 67 445 91 234 8 56 178 33 902 44 7 123 55 678 21 9 333 87
zsh: segmentation fault  ./menu
```

Conclusione

Il laboratorio ha permesso di comprendere concretamente il funzionamento di una vulnerabilità di tipo Buffer Overflow, partendo dall'analisi teorica del codice fino alla sua dimostrazione pratica. Attraverso la modifica del programma originale e lo sviluppo della versione con menù, è stato possibile osservare direttamente come la mancanza di controlli sull'input utente possa portare a comportamenti imprevedibili e al crash del programma.

```
Inserire interi (premi INVIO alla fine):
2 4 5 6 5 4 3 2 3  34 56 6 5 4 3 2 3 44 5 56 4  43
zsh: segmentation fault  ./bof.02
```